

EE2801 - DSP LAB

EXPERIMENT-2

Fixed-Point Arithmetic using Verilog

EE24BTECH11012 - BHAVANISANKAR G S

February 7, 2026

Contents

1	Plots	2
2	Observations	4
3	Conclusions	4
4	Codes	5

1 Plots

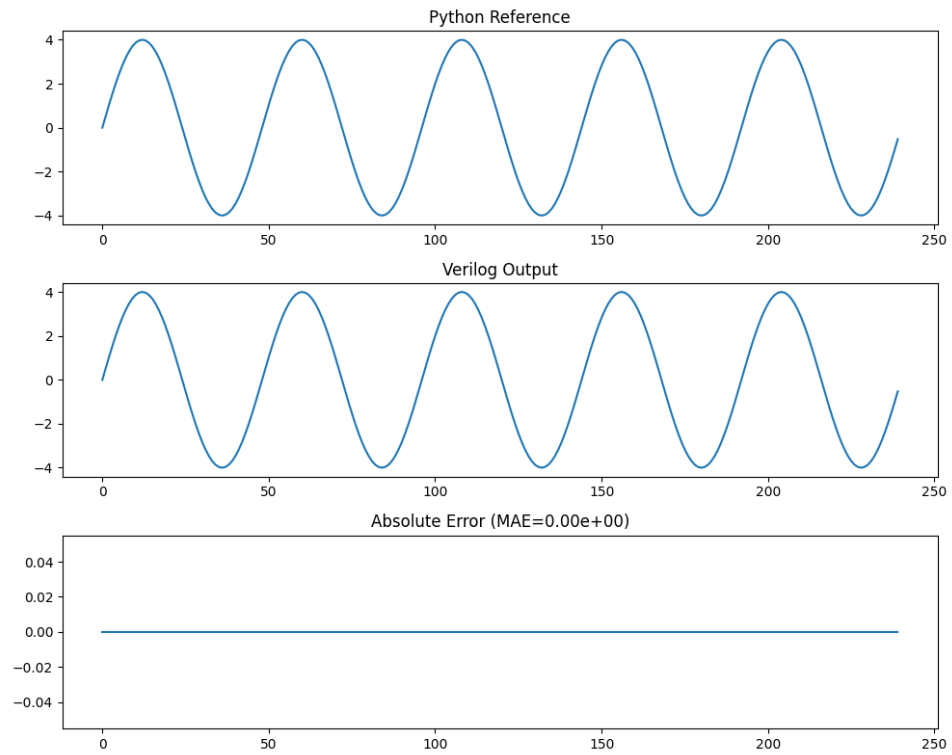


Figure 1: Addition results

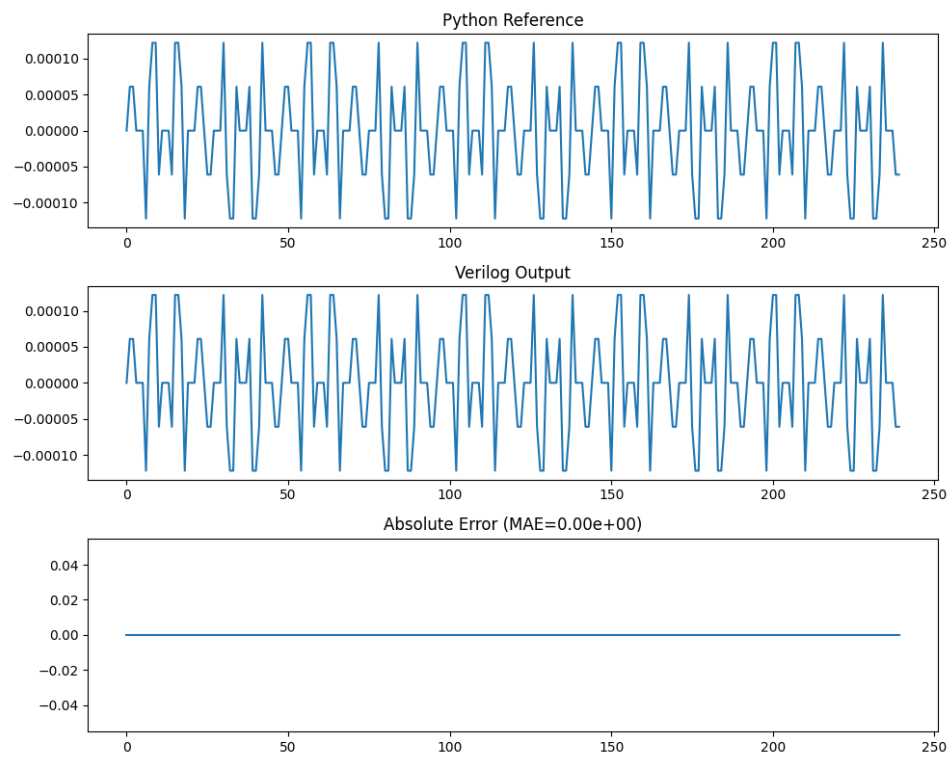


Figure 2: Subtraction results

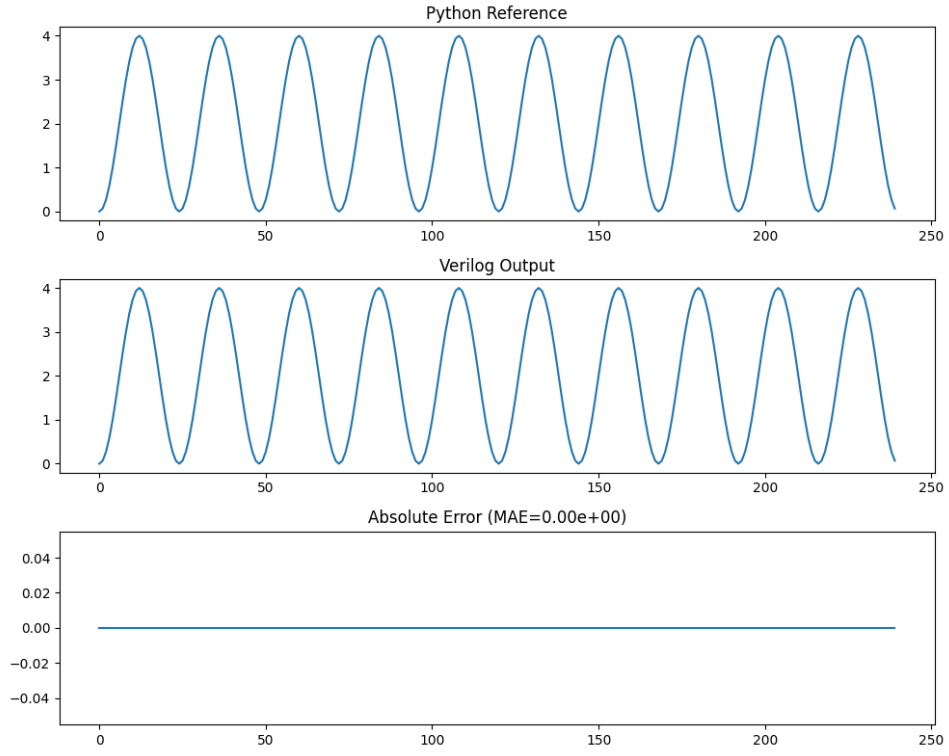


Figure 3: Multiplication results in Q(10, 28) format

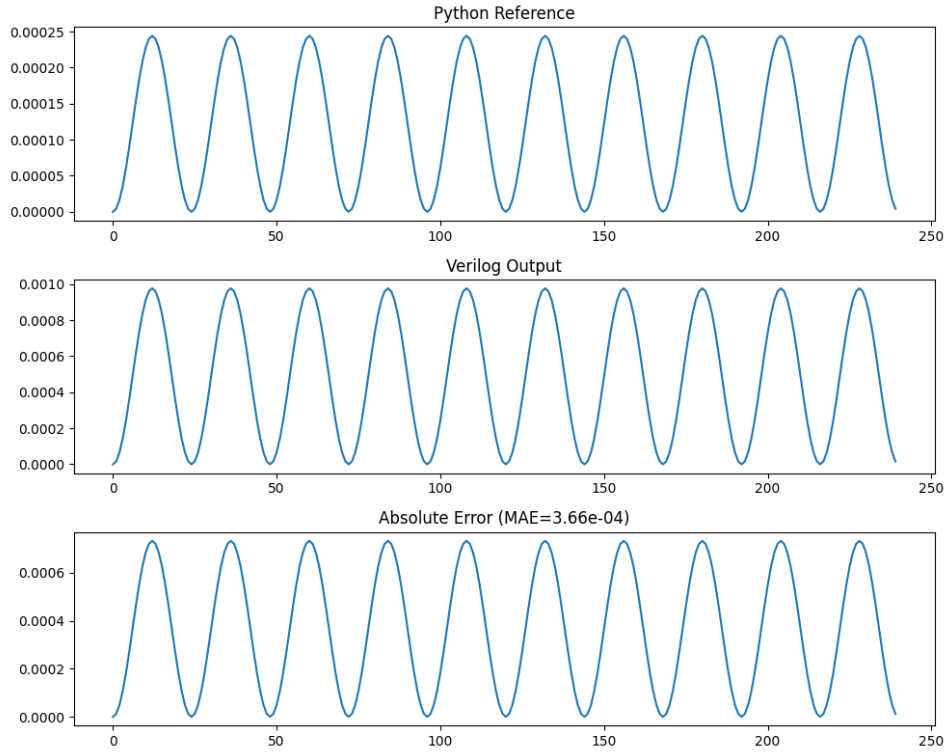


Figure 4: Multiplication results in Q(10, 14) format - Hardware Efficient

2 Observations

```
Plots saved in 'figs/'
(venv) bhavan@bhavan-ubuntu:~/Academics/Sem-4/DSP Lab/Experiment-2$ python3 verify.py
-----
Mean Absolute Error
-----
ADD | MAE = 0.000e+00
SUB | MAE = 0.000e+00
MUL | MAE = 0.000e+00
MUL_EFF | MAE = 3.662e-04
-----
Plots saved in 'figs/'
```

Figure 5: Errors obtained comparing Python and Verilog outputs

- Fixed-point addition and subtraction operations produced outputs that exactly matched the Python floating-point reference model, resulting in zero mean absolute error (MAE).
- Subtraction of two signals derived from the same sinusoidal waveform resulted in a small-amplitude output, which is expected due to minor fixed-point quantization differences between operands.
- Full-precision fixed-point multiplication, retaining the complete fractional growth ($Q(10,28)$), showed zero numerical error when compared with the Python reference.
- A hardware-efficient multiplication approach (MUL_EFF), obtained by right-shifting the full-precision product by $\text{FRAC}_{\max}$ bits to restore the original fractional width ($Q(10,14)$), introduced a small bounded quantization error.
- The measured mean absolute error for the MUL_EFF operation was 3.662×10^{-4} , consistent with theoretical truncation error limits.

3 Conclusions

- Fixed-point arithmetic can accurately replicate floating-point computations when proper Q-format alignment and scaling are applied.
- Full-precision multiplication ensures bit-exact accuracy but incurs increased hardware cost due to significant bit-width growth.
- Truncating the multiplier output to limit fractional precision introduces a predictable and bounded quantization error while substantially improving hardware efficiency.

- The MUL_EFF approach provides an effective trade-off between numerical accuracy and FPGA resource utilization, making it suitable for real-time DSP applications.
- The close agreement between Python reference results and Verilog outputs validates the correctness and robustness of the fixed-point ALU implementation.

4 Codes

Listing 1: MATLAB code for Signal Generation

```

1  clc ;
2  close all ;
3  clear all ;
4
5  N = 240 ; % No of samples
6  f = 1e3 ;
7  fs = 10e3 ;
8  t = 0:1/fs:(5e-2 - 1/fs) ;
9
10 x = 2 * sin(2 * pi * f * t) ;
11 total_len = 17 ;
12 frac_len = 14 ;
13 fixed = fi(x, 1, total_len, frac_len) ;
14
15 filename = 'files/op1_bin.txt' ;
16 file = fopen(filename, 'w') ;
17
18 fprintf(file, "// %d %d\n", total_len - frac_len, frac_len) ;
19 for k = 1:length(x)
20     fprintf(file, "%s\n", bin(fixed(k))) ;
21 end
22
23 fprintf('Data generated for Q(%d, %d) and written into the file %s\n', total_len -
    frac_len, frac_len, filename) ;

```

Listing 2: Verilog testbench module for printing values into a file

```

1  // W - Width of the fixed-point number
2  // FRAC - No of fractional bits
3  module align_frac #(
4      parameter W_IN = 17,
5      parameter W_OUT = 19,
6      parameter FRAC_IN = 14,
7      parameter FRAC_OUT = 14
8  ) (
9      input signed [W_IN-1:0] in,
10     output signed [W_OUT-1:0] out
11 );
12
13 wire signed [W_OUT-1:0] temp ;
14 assign temp = in ;
15
16 parameter SHIFT = FRAC_OUT - FRAC_IN ;
17 assign out = (SHIFT > 0) ? (temp << SHIFT) : ((SHIFT) ? (temp >>> (-SHIFT)) : temp) ;
18

```

```

19 endmodule
20
21 // Module to perform fixed-point addition
22 module fxp_add #(
23     parameter WIDTH = 17
24 ) (
25     input  signed [WIDTH-1:0] a,
26     input  signed [WIDTH-1:0] b,
27     output signed [WIDTH-1:0] out
28 );
29
30     assign out = a + b ;
31
32 endmodule
33
34 // Module to perform fixed-point subtraction
35 module fxp_sub #(
36     parameter WIDTH = 17
37 ) (
38     input  signed [WIDTH-1:0] a,
39     input  signed [WIDTH-1:0] b,
40     output signed [WIDTH-1:0] out
41 );
42
43     assign out = a - b ;
44
45 endmodule
46
47 // Module to perform fixed-point multiplication
48 module fxp_mul #(
49     parameter WIDTH = 17
50 ) (
51     input  signed [WIDTH-1:0] a,
52     input  signed [WIDTH-1:0] b,
53     output signed [2*WIDTH-1:0] out
54 );
55
56     assign out = a * b ;
57
58 endmodule
59
60 module fxp_alu #(
61     parameter Q_IN1 = 3,
62     parameter Q_IN2 = 5,
63     parameter FRAC_IN1 = 14,
64     parameter FRAC_IN2 = 12,
65     parameter WIDTH = 19
66 ) (
67     input  signed [Q_IN1+FRAC_IN1-1:0] op1,
68     input  signed [Q_IN2+FRAC_IN2-1:0] op2,
69     output signed [WIDTH-1:0] op_add,
70     output signed [WIDTH-1:0] op_sub,
71     output signed [2*WIDTH-1:0] op_mul,
72     output signed [Q_MAX+FRAC_MAX-1:0] mul_eff
73 );
74
75     parameter Q_MAX = ( Q_IN1 > Q_IN2 ) ? Q_IN1 : Q_IN2 ;
76     parameter FRAC_MAX = ( FRAC_IN1 > FRAC_IN2 ) ? FRAC_IN1 : FRAC_IN2 ;
77     parameter W_ALIGNED = Q_MAX + FRAC_MAX ;

```

```

78     parameter W_IN = Q_IN1 + FRAC_IN1 ;
79
80     wire signed [W_ALIGNED-1:0] op1_aligned, op2_aligned ;
81     // wire signed [Q_MAX_FRAC_MAX-1:0] mul_eff ;
82
83     align_frac #(W_IN, W_ALIGNED, FRAC_IN1, FRAC_MAX) OP1(op1, op1_aligned) ;
84     align_frac #(W_IN, W_ALIGNED, FRAC_IN2, FRAC_MAX) OP2(op2, op2_aligned) ;
85
86     // fxp_add #(W_ALIGNED) addition(op1_aligned, op2_aligned, op_add) ;
87     // fxp_sub #(W_ALIGNED) subtraction(op1_aligned, op2_aligned, op_sub) ;
88     // fxp_mul #(W_ALIGNED) multiplication(op1_aligned, op2_aligned, op_mul) ;
89
90     adder_subtractor #(W_ALIGNED) adder(op1_aligned, op2_aligned, 1'b0, 1'b0, op_add) ;
91     adder_subtractor #(W_ALIGNED) subtractor(op1_aligned, op2_aligned, 1'b1, 1'b1,
92         op_sub) ;
93     multiplier #(W_ALIGNED) multiply(op1_aligned, op2_aligned, op_mul) ;
94     assign mul_eff = op_mul >>> FRAC_MAX ;
95
96 endmodule
97
98 module adder_subtractor #(
99     parameter WIDTH = 17
100 )(
101     input signed [WIDTH-1:0] a,
102     input signed [WIDTH-1:0] b,
103     input cin,
104     input ctrl,
105     output reg signed [WIDTH-1:0] sum
106     // output cout
107 );
108
109     wire signed [WIDTH-1:0] b_transformed = b ^ {WIDTH{ctrl}} ;
110     reg signed [WIDTH-1:0] carry ;
111     integer i ;
112     always @(*) begin
113         sum[0] = a[0] ^ b_transformed[0] ^ cin ;
114         carry[0] = ( a[0] & b_transformed[0] ) | ( b_transformed[0] & cin ) | ( a[0] &
115             cin ) ;
116
117         for ( i = 1 ; i < WIDTH ; i += 1 ) begin
118             sum[i] = a[i] ^ b_transformed[i] ^ carry[i-1] ;
119             carry[i] = ( a[i] & b_transformed[i] ) | ( b_transformed[i] & carry[i-1] ) |
120                 ( a[i] & carry[i-1] ) ;
121         end
122     end
123
124     // assign cout = carry[WIDTH-1] ;
125 endmodule ;
126
127 module multiplier #(
128     parameter WIDTH = 17
129 )(
130     input signed [WIDTH-1:0] a,
131     input signed [WIDTH-1:0] b,
132     output reg signed [2*WIDTH-1:0] out
133 );
134
135     reg signed [WIDTH-1:0] a_temp ;
136     reg signed [WIDTH-1:0] b_temp ;

```

```

134     reg sign ;
135     reg signed [WIDTH:0] q ;
136     integer i ;
137     always @(*) begin
138
139         a_temp = (a[WIDTH-1]) ? -a : a ;
140         b_temp = (b[WIDTH-1]) ? -b : b ;
141         sign = a[WIDTH-1] ^ b[WIDTH-1] ;
142
143         q = 0 ;
144         for ( i = 0 ; i < WIDTH ; i += 1 ) begin
145             q = (a_temp[0]) ? (q + {1'b0, b_temp}) : q ;
146             {q, a_temp} = {1'b0, q, a_temp[WIDTH-1:1]} ;
147         end
148         out = (sign) ? -{q[WIDTH-1:0], a_temp} : {q[WIDTH-1:0], a_temp};
149     end
150
151 endmodule ;

```

Listing 3: Verilog module for performing arithmetic

```

1  `timescale 1ns/1ps
2
3  module tb_fxp_alu ;
4
5      parameter N = 240 ;
6      parameter Q1 = 3, Q2 = 5 ;
7      parameter FRAC1 = 14, FRAC2 = 12 ;
8      parameter WIDTH = Q1 + FRAC1 ;
9
10     parameter Q_MAX = (Q1 > Q2) ? Q1 : Q2 ;
11     parameter FRAC_MAX = (FRAC1 > FRAC2) ? FRAC1 : FRAC2 ;
12     parameter WIDTH_ALIGNED = Q_MAX + FRAC_MAX ;
13
14
15     integer i ;
16     integer f_add, f_sub, f_mul, f_mul_eff ;
17
18     reg signed [WIDTH-1:0] op1_mem [0:N-1] ;
19     reg signed [WIDTH-1:0] op2_mem [0:N-1] ;
20     reg signed [WIDTH-1:0] op1, op2 ;
21
22     wire signed [WIDTH_ALIGNED-1:0] op_add ;
23     wire signed [WIDTH_ALIGNED-1:0] op_sub ;
24     wire signed [2*WIDTH_ALIGNED-1:0] op_mul ;
25     wire signed [Q_MAX+FRAC_MAX-1:0] mul_eff ;
26
27     fxp_alu #(Q1, Q2, FRAC1, FRAC2, WIDTH_ALIGNED) uut(op1, op2, op_add, op_sub, op_mul,
28         mul_eff) ;
29
30     initial begin
31         $readmemb("files/op1_bin.txt", op1_mem) ;
32         $readmemb("files/op2_bin.txt", op2_mem) ;
33
34         f_add = $fopen("files/add_outputs.txt", "w") ;
35         f_sub = $fopen("files/sub_outputs.txt", "w") ;
36         f_mul = $fopen("files/mul_outputs.txt", "w") ;
37         f_mul_eff = $fopen("files/mul_eff_outputs.txt", "w") ;

```



```

38     for ( i = 0 ; i < N ; i += 1 ) begin
39         op1 = op1_mem[i] ;
40         op2 = op2_mem[i] ; # 1 ;
41
42         $fwrite(f_add, "%d\n", op_add) ;
43         $fwrite(f_sub, "%d\n", op_sub) ;
44         $fwrite(f_mul, "%d\n", op_mul) ;
45         $fwrite(f_mul_eff, "%d\n", mul_eff) ;
46     end
47
48     $fclose(f_add) ;
49     $fclose(f_sub) ;
50     $fclose(f_mul) ;
51     $fclose(f_mul_eff) ;
52     $finish ;
53
54 end
55
56 endmodule

```

Listing 4: Python code for verification

```

1  import numpy as np
2  import matplotlib.pyplot as plt
3  import os
4
5  DATA = "files"
6  FIGS = "figs"
7  WIDTH = 17
8
9  os.makedirs(FIGS, exist_ok=True)
10
11 def load_bin(name):
12     with open(f"{DATA}/{name}.txt") as f:
13         _, ib, fb = f.readline().split()
14         fb = int(fb)
15         bits = f.read().split()
16
17         ints = np.array([int(b, 2) - (1 << WIDTH) if b[0] == '1' else int(b, 2) for b in
18             bits])
19
20         return ints / (2 ** fb), fb
21
22 def load_verilog(name, fb):
23     ints = np.loadtxt(f"{DATA}/{name}_outputs.txt", dtype=int)
24     if name == 'mul' :
25         return ints / (4 * (2 ** fb))
26     else :
27         return ints / (2 ** fb)
28
29
30
31 def plot(op, ref, ver):
32     err = np.abs(ver - ref)
33     mae = np.mean(err)
34
35     print(f"{op.upper():>4} | MAE = {mae:.3e}")
36

```

```

37     plt.figure(figsize=(10, 8))
38
39     plt.subplot(3, 1, 1)
40     plt.plot(ref)
41     plt.title("Python Reference")
42
43     plt.subplot(3, 1, 2)
44     plt.plot(ver)
45     plt.title("Verilog Output")
46
47     plt.subplot(3, 1, 3)
48     plt.plot(err)
49     plt.title(f"Absolute Error (MAE={mae:.2e})")
50
51     plt.tight_layout()
52     plt.savefig(f"{FIGS}/{op}.png")
53     plt.close()
54
55
56 op1, fb1 = load_bin("op1_bin")
57 op2, fb2 = load_bin("op2_bin")
58
59 results = {
60     "add": (op1 + op2, max(fb1, fb2)),
61     "sub": (op1 - op2, max(fb1, fb2)),
62     "mul": (op1 * op2, fb1 + fb2),
63     "mul_eff" : (op1 * op2/(2 ** 14), fb1 + fb2),
64 }
65
66 print("-" * 30)
67 print("Mean Absolute Error")
68 print("-" * 30)
69
70 for name, (ref, fb) in results.items():
71     ver = load_verilog(name, fb)
72     plot(name, ref, ver)
73
74 print("-" * 30)
75 print(f"Plots saved in '{FIGS}/'")

```